

```
[Рабочая папка] --git add--> [Индекс (staging)] --git commit--> [Локальный репозиторий .git] --git push-->
```

<--git

2. Что установлено на вашем компьютере

Инструмент	Что это	Где / как вызвать
Git 2.54	Основная программа	C:\Program Files\Git\cmd\git.exe , консоль <code>git ...</code>
Git Bash Here	Консоль Git (Linux-подобная)	Правый клик в папке → «Git Bash Here»
Git GUI Here	Графический интерфейс Git	Правый клик в папке → «Git GUI Here»
TortoiseGit	Правый клик: Git Pull/Push/Commit/Status, значки-оверлеи	Правый клик в папке; значок «черепашка»
Git Extensions	Отдельное графическое приложение для Git	Правый клик → «Git Extensions»
WinMerge	Сравнение и слияние файлов (конфликты)	Правый клик → «WinMerge»
TortoiseSVN	Клиент SVN (другая система контроля версий)	Правый клик → «SVN Checkout», «TortoiseSVN»
GitHub CLI (<code>gh</code>)	Работа с GitHub из консоли	не установлен (можно поставить отдельно)

Итог: у вас есть **и консольный Git, и графические инструменты** (TortoiseGit, Git GUI, Git Extensions).

Для повседневной работы удобнее всего **TortoiseGit** (правый клик) или **Git Extensions** (визуальная история).

3. Ваши проекты и репозитории

Папка	Что это	Remote (GitHub)	Ветка	Состояние
C:\AIS\AI\Prod	Главный проект PROD	<code>https://github.com/Chagav1958/PROD.git</code>	<code>main</code>	синхронизирован с origin

C:\AIS\AI\Prod_clean	Чистая копия PROD	https://github.com/Chagav1958/PROD.git	main	есть коммиты
C:\AIS\AI\MCP_EWS	Сервер MCP для Exchange	https://github.com/chagav/mcp_ews.git	master	связан с GitHub
C:\AIS\AI\SyBooks	Документация Sybase	— (только локально)	main	локальный
C:\AIS\AI\PB_LSP	LSP-сервер PowerBuilder	— (только локально)	master	локальный
C:\AIS\AI\Visualization\PB_LSP	Копия PB_LSP	— (только локально)	master	локальный
C:\AIS\AI\AIS , Service , BAD\OLD_BAD_Step1	Черновики	—	master	без коммитов

Главный рабочий репозиторий — C:\AIS\AI\Prod, СВЯЗАННЫЙ С github.com/Chagav1958/PROD.

У вас два аккаунта GitHub: **Chagav1958** (репозиторий PROD) и **chagav** (репозиторий mcp_ews).

4. Уже настроено (проверено)

- Имя и e-mail для коммитов заданы глобально:

- user.name = Valery chagav

- user.email = chagav58@gmail.com

- Основная ветка главного репозитория — main, привязана к origin/main (отслеживается).

- В C:\AIS\AI\Prod есть файл .gitignore с политикой «игнорировать всё в корне, кроме явно разрешённого»

(первая строка /*) — так секреты и большие файлы не попадают в GitHub.

- Действует правило безопасности: логины/пароли/токены хранятся только в

C:\AIS\AI\Prod\config\local_secrets.json (ОН В .gitignore).

5. Основные команды Git (с примерами на вашем проекте)

> Все команды ниже выполняются в консоли **Git Bash** (правый клик в папке `C:\AIS\AI\Prod` → «Git Bash Here»).

> В примерах используется ваш главный проект.

5.1. Где я и что изменилось

```
cd /c/AIS/AI/Prod      # перейти в папку проекта (в Git Bash диск указывается как /c/)

git status              # показать: ветка, изменённые/новые файлы

git status -s           # короткий формат
```

Пример вывода: `On branch main`, далее список `modified:` и `untracked:`.

5.2. Посмотреть историю

```
git log --oneline -10    # последние 10 коммитов (коротко)

git log --oneline --graph --all  # история веток в виде графа

git show addabcb         # подробно показать коммит по его хэшу (первые 7 символов)
```

У вас, например: `addabcb` Новый сервис, `576fcf4` Файл правил для сокращения.

5.3. Что именно изменилось в файлах

```
git diff                                # изменения ещё не добавленных в индекс файлов

git diff --stat                         # только сводка (какие файлы и на сколько строк)

git diff HEAD -- docs/git-github-guide.md # изменения конкретного файла
```

5.4. Сохранить изменения (коммит)

```
git add .                               # добавить ВСЕ изменения в индекс (staging)

git add docs/git-github-guide.md        # или добавить конкретный файл

git commit -m "Docs: руководство по Git и GitHub" # зафиксировать снимок
```

Правила хорошего коммита:

- один коммит — одна логическая правка;
- сообщение на русском, кратко и по делу (какие изменения и зачем);
- **никогда** не коммитить секреты (файл `.local_secrets.json` и т.п.).

5.5. Отправить на GitHub (push)

```
git push                                # отправить коммиты в origin/main

git push -u origin main                 # первый раз для новой ветки (задать отслеживание)
```

5.6. Получить изменения с GitHub (pull)

```
git pull                                # скачать и влить изменения из origin/main

git fetch                               # только скачать (без вливания), потом посмотреть: git log HEAD..origin/main
```

5.7. Ветки

```
git branch                # список локальных веток (* – текущая)

git switch -c feature-new-service # создать ветку и переключиться (или: git checkout -b ...)

git switch main           # вернуться на main

git branch -d feature-new-service # удалить ветку (после слияния)
```

Слияние ветки в main:

```
git switch main

git pull

git merge feature-new-service

git push
```

5.8. Отменить изменения

```
git restore docs/file.md      # отбросить правки в файле (вернуть как было в последнем коммите)

git restore --staged docs/file.md # убрать файл из индекса (после «git add»)

git revert <хэш>               # создать коммит, отменяющий указанный коммит (безопасно)

git reset --soft HEAD~1       # снять последний коммит, сохранив изменения в рабочей папке
```

Важно: `git reset --hard` **уничтожает** изменения — применять только осознанно.

5.9. Клонирование (получить проект на новый ПК)

```
cd /c/AIS/AI

git clone https://github.com/Chagav1958/PROD.git PROD_new
```

5.10. Полезное

```
git remote -v          # показать адреса удалённых репозиторий
git remote show origin  # подробно об origin (нужна сеть)
git tag -a v1.0 -m "Версия 1.0" # поставить тег (метку версии)
git push --tags         # отправить теги на GitHub
```

6. Графические инструменты (без консоли)

6.1. TortoiseGit (правый клик по папке)

- **Git Commit** → **main...** — выбрать изменённые файлы, написать сообщение, нажать «Commit».
- **Git Pull** — получить изменения с GitHub.
- **Git Push** — отправить свои коммиты.
- **Git Status** — окно со списком изменённых файлов.
- **Git Log** — просмотр истории (можно правый клик → «Show Log» прямо в папке).
- **Diff** (двойной клик по файлу в окне Commit/Log) — сравнение версий.
- Иконки-оверлеи на файлах: зелёная галочка — без изменений, красный — изменён, «+» — новый.

Порядок отправки изменений через TortoiseGit:

1. Правый клик по `C:\AIS\AI\Prod` → **Git Commit** → **main....**
2. Отметить галочками нужные файлы, ввести сообщение, «Commit».
3. Правый клик → **Git Push** → «ОК».

6.2. Git Extensions (визуальный клиент)

Правый клик по папке → **Git Extensions** → открывается окно с историей, ветками, файлами.

Здесь удобно: смотреть граф коммитов, делать Commit/Pull/Push кнопками, разрешать конфликты.

6.3. WinMerge (слияние конфликтов)

Если при `git pull` возник конфликт, файл помечается как «конфликтующий». Откройте его в WinMerge (правый клик → WinMerge, либо TortoiseGit → Edit conflicts), выберите нужные строки, сохраните файл, затем:

```
git add конфликтный-файл.md  
  
git commit -m "Разрешён конфликт"
```

Альтернатива SVN (если где-то используется): правый клик → **SVN Checkout / TortoiseSVN** — это другая система (Subversion), принцип похож (checkout/update/commit), но команды отличаются.

7. GitHub: что можно делать на сайте

Адрес: `https://github.com/Chagav1958/PROD` (ваш главный репозиторий).

Возможность	Что даёт
Repository (репозиторий)	Хранение кода и истории, кнопки «Code» (ссылка для clone)
README.md	Главная страница описания проекта (отображается на странице репозитория)
Issues (задачи)	Список задач/ошибок с привязкой к обсуждению
Pull Requests (PR)	Предложение изменений (из ветки в main), обзор и слияние
Releases (релизы)	Помеченные версии с описанием и вложениями (файлы сборок)
Actions (автоматизация)	Автосборка/тесты при коммите (CI/CD)
Pages (страницы)	Публикация статичного сайта из репозитория
Commits / Blame	История изменений файла и авторство каждой строки
Settings → Collaborators	Доступ других людей к репозиторию
Settings → Deploy keys / Tokens	Доступ по SSH-ключу или токену (для автоматизации)

Работа с сайтом (веб):

- Посмотреть изменения: открыть файл → кнопка «History».
- Создать задачу: вкладка **Issues** → «New issue».
- Сравнить ветки и сделать PR: вкладка **Pull requests** → «New pull request».

8. Безопасность (важно!)

1. **Секреты только в** `.local_secrets.json` (`C:\AIS\AI\Prod\config\.local_secrets.json`).

Этот файл добавлен в `.gitignore` и **никогда** не попадает в Git/GitHub.

2. В проекте реализован **pre-commit hook** и утилита очистки (`Clean-GitSecrets`) — они не дают закоммитить пароли/токены. Если Git отказал в коммите — значит найден секрет, уберите его.

3. **Personal Access Token (PAT)** для GitHub: создаётся на сайте (Settings → Developer settings → Personal access tokens). Хранить токен — только в защищённом месте (не в файлах репозитория).
4. Регулярно проверяйте `git status` перед push: не попадают ли лишние (большие/секретные) файлы.
5. Правило проекта: `config\config.json` и `config\settings_history.json` — без секретов; значения подстановочные (`***`), сами секреты — в `.local_secrets.json`.

9. Типовые сценарии (пошагово)

Сценарий А. «Я изменил файлы и хочу сохранить их на GitHub»

```
cd /c/AIS/AI/Prod

git status          # посмотреть, что изменилось

git add .           # добавить изменения

git commit -m "Краткое описание правок"

git push
```

Сценарий В. «Я хочу получить свежую версию с GitHub перед работой»

```
cd /c/AIS/AI/Prod

git pull
```

Сценарий С. «Хочу вернуться к состоянию до правок»

```
git restore путь_к_файлу    # один файл

# или откатить всё (осторожно):

git restore .
```

Сценарий D. «Хочу посмотреть, что было в прошлом коммите»

```
git log --oneline -10

git show <хэш_коммита>
```

Сценарий E. «Первая публикация локального проекта на GitHub»

```
cd /c/AIS/AI/MyProject

git init

git add .

git commit -m "Первый коммит"

git branch -M main

git remote add origin https://github.com/ВАШ_ЛОГИН/ИМЯ_РЕПОЗИТОРИЯ.git

git push -u origin main
```

10. Шпаргалка (одной строкой)

```
git status                # состояние

git pull                  # получить
```

```
git add .                # подготовить всё

git commit -m "текст"    # сохранить

git push                 # отправить

git log --oneline -10    # история

git diff                 # что изменилось

git restore файл         # отменить правки файла

git branch -vv           # ветки и их привязка

git clone URL            # скопировать репозиторий
```

11. Частые вопросы и ошибки

Ситуация	Что делать
<code>fatal: not a git repository</code>	Вы не в папке проекта: <code>cd</code> в папку с <code>.git</code>
<code>Permission denied (publickey)</code> при <code>push</code>	Нет доступа: проверьте токен/GitHub-аккаунт, права на репозиторий
<code>Updates were rejected (push)</code>	Сначала <code>git pull</code> , разрешите конфликт, затем <code>git push</code>
Случайно закоммитил секрет	Отзовите (смените) секрет на сервере, удалите файл из Git и истории (утилита очистки проекта)
Конфликт при <code>pull</code>	Откройте конфликтные файлы (WinMerge), выберите строки, <code>git add</code> , <code>git commit</code>
Хочу отменить последний (не отправленный) коммит	<code>git reset --soft HEAD~1</code> (изменения останутся в папке)
Большой файл не нужен в Git	Добавьте его в <code>.gitignore</code>

12. Ссылки и дальнейшее чтение

- Официальная документация Git (русский): <https://git-scm.com/book/ru/v2>
 - Справка GitHub: <https://docs.github.com/ru>
 - Ваш репозиторий: <https://github.com/Chagav1958/PROD>
 - Локальная справка: в Git Bash команда `git help <команда>` (например, `git help commit`).
 - Инструмент GitHub CLI (`gh`) для работы из консоли (по желанию): <https://cli.github.com>
- > Совет: начните с **TortoiseGit** (правый клик → Git Commit/Pull/Push) — это самый простой способ
- > без командной строки. По мере освоения переходите к командам из разделов 5 и 10.